

PD 제어 가이드 (라인 트레이서)

이 문서는 라인 트레이서의 **PD 제어**가 무엇이고, 왜 P·D만 쓰는지, 어떻게 튜닝하는지를 예시 중심으로 설명합니다. 설정값·증상별 표 같은 빠른 레퍼런스는 [README.md §4.4](#)를 보세요. 여기서는 ****원리와 감(感)****을 잡는 게 목적입니다.

PD 제어 본체는 `LineTrace()`, 상수는 `Settings_robot1.h:107-108`에 있습니다.

```
int error = rightNorm - leftNorm;    // 라인이 좌/우로 얼마나 틀어졌나 (범위 대략 -1000 ~ +1000)
int dError = error - _prevError;      // 직전 루프와의 변화량 (de/dt 가 아니라 "한 틱 차이")
float correction = Kp * error + Kd * dError;
_prevError = error;
```

`error`가 +면 라인이 오른쪽으로 치우친 상태(오른쪽 센서가 검은선을 더 봄)라고 가정합니다.

1. P / I / D 가 각각 보는 "시간"

PID 세 항은 같은 오차를 **다른 시간 관점**으로 봅니다.

항	보는 것	이 코드
P (비례)	지금 오차	<code>Kp * error</code> — 지금 틀어진 만큼 바로 꺾음
I (적분)	과거 오차의 누적합 ($\int error$)	없음 (아래 §4 참고)
D (미분)	오차의 변화 속도	<code>Kd * dError</code> — 빠르게 틀어지면 미리 잡음 (진동 억제)

2. 예시로 따라가기 — D항이 왜 필요한가

로봇이 중앙에서 **오른쪽으로 점점 빠르게 벗어나는** 장면. 루프가 빠르니 한 틱마다 `error`가 조금씩 커집니다.

(1) 벗어나는 국면

틱	error	dError	상황
1	0	0	중앙
2	20	+20	막 틀어지기 시작
3	50	+30	빠르게 벗어나
4	90	+40	더 빠르게 벗어나

틱 4 보정량 (`Kp=0.04`, `Kd=0.3`):

$P\text{항} = 0.04 \times 90 = 3.6$ ← "지금 90 들어졌으니 3.6 꺾어"
 $D\text{항} = 0.3 \times 40 = 12.0$ ← "점점 더 빨리(+40) 벗어나는 중이네, 12 더 세게 미리 꺾어!"
 $\text{correction} = 3.6 + 12.0 = 15.6$

- **P항(3.6)** 은 현재 위치만 본다 → 작다.
- **D항(12.0)** 은 추세를 본다 → 아직 많이 안 벗어났는데도 미리 강하게 잡는다. 이게 지그재그를 막는 댐핑(damping).

(2) 되돌아오는 국면 — D항이 브레이크를 건다

틱	error	dError	P항	D항
5	90	0	3.6	0
6	60	-30	2.4	-9.0
7	30	-30	1.2	-9.0

error는 아직 +(오른쪽 치우침)이라 **P항은 계속 "더 왼쪽으로 꺾어"** 라고 한다. 그런데 **D항은 -9.0** — "이미 빠르게 중앙으로 돌아오는 중이니 그만 꺾어, 안 그러면 또 지나친다(오버슈트)" 하고 **브레이크**를 건다.

P 혼자였다면 중앙을 지나쳐 왼쪽으로 넘어가고, 다시 지그재그가 났을 것이다. → D항이 충격흡수기 역할.

3. 왜 Kd 가 Kp 보다 5~30배 큰가

"5~30배"는 물리 법칙이 아니라, 이렇게 짜인 이산(discrete) 컨트롤러에서 두 항의 숫자 크기를 비슷하게 맞추면 자연스럽게 나오는 범위입니다. 이유 두 가지.

이유 1 (핵심): error 와 dError 의 크기가 애초에 다르다

dError는 수학적 미분(de/dt)이 아니라 루프 한 바퀴 사이의 변화량입니다 (dt로 안 나눔). 루프가 수 ms마다 도는 빠른 루프라 연속 두 샘플 사이 **error**는 조금밖에 안 변합니다.

$|\text{error}| \approx \text{수십} \sim \text{수백}$
 $|\text{dError}| \approx \text{한 자리} \sim \text{십 단위}$ ← 한 틱 변화라 훨씬 작음

D항이 P항과 비슷한 영향력을 가지려면 작은 **dError**에 곱하는 **Kd**를 그만큼 키워야 합니다. 그래서 **Kd > Kp**가 거의 강제 돼요.

실제 값(**Kp=0.04**, **Kd=0.3**, 7.5배)으로, **error=200**·**dError=20**이라 가정하면:

$P\text{항} = 0.04 \times 200 = 8.0$
 $D\text{항} = 0.3 \times 20 = 6.0$ ← 둘이 비슷! 의도대로 균형이 맞음

만약 $K_d = K_p$ 였다면 D항은 $0.04 \times 20 = 0.8$ 로 P의 1/10밖에 안 돼서 댐핑이 사실상 사라집니다. 곱하는 배수 = "dError가 error보다 작은 만큼"을 메꿔주는 보정인 셈. (루프가 느려 dt가 컸다면 dError가 커지고 Kd는 작아도 됐을 것 → 절대값이 아니라 비율이 의미 있는 이유.)

이유 2 (제어이론): 라인트레이서는 "잘 흔들리는" 시스템

- P 단독은 모터 응답 지연 + 차체 관성 때문에 늘 늦게 반응 → 중앙을 지나쳐 반대편으로, 또 반대편으로... **저댐핑 (under-damped) = 진동.**
- D항이 그 진동을 눌러 **임계감쇠(critically damped)**에 가깝게 만든다. 그러려면 댐핑 항이 비례 항보다 충분히 세야 한다.

⚠ 단, D항은 노이즈를 증폭한다(미분은 잡음에 민감). 그래서 Kd를 무한정 못 키운다 → "30배 근처"가 상한 가이드로 잡히는 이유.

4. 왜 I항(적분)은 안 쓰는가

I항은 작은 오차가 오래 지속될 때 그걸 누적($\int error$)해서 마저 0으로 밀어붙이는 항입니다. 상시 드리프트·정상상태 오차 제거가 특기. 이 라인트레이서엔 거의 무용하고 오히려 해롭습니다.

1. 누적될 시간이 없다. error 부호가 좌우로 계속 바뀌어 +오차와 -오차가 적분에서 상쇄됨.
2. 교차로마다 리셋된다. 교차로 진입/이탈 시 `_prevError = 0` 처리([Controller.cpp:661, 672](#)). 한 칸씩 끊어 달리는 구조라 누적이 쌓일 틈이 없음.
3. 적분 와인드업(windup) 위험. 곡선·통과 구간에서 I항이 부풀었다가 늦게 풀리며 오히려 오버슈트.
4. 드리프트는 이미 따로 잡는다. 한쪽 쓸림은 모터 캘리브레이션(`_motorCalibL/R`, EEPROM)이 하드웨어 단에서 보정 → I항이 메꿀 자리를 이미 차지.

결론: 라인트레이서는 업계에서도 보통 **PD 제어**가 표준입니다. 한쪽으로 꾸준히 휘면 I를 넣지 말고 **모터 캘리브 재실행**으로 잡으세요.

5. 튜닝하는 법

스텝은 "덧셈"이 아니라 "곱셈(배수)"으로

값이 작아서($K_p=0.04$) $+0.01$ 하면 +25%로 사실 큰 변화입니다. "얼마 더하나"가 아니라 "몇 % 바꾸나"로 생각하세요. 10% 조정은 현재 값에 $\times 1.1$ / $\times 0.9$:

상수	현재값	$\times 1.1$ (+10%)	$\times 0.9$ (-10%)
K_p	0.04	0.044	0.036
K_d	0.3	0.33	0.27

각자 자기 값의 %라서 K_p 는 0.004 단위, K_d 는 0.03 단위로 움직입니다 (폭이 다른 게 정상).

큰 폭 → 작은 폭 순서로

처음부터 10%씩 가면 차이를 못 느껴 오래 걸립니다. 크게 **흔들어 방향(올려야/내려야 좋아지나)**부터 찾고 점점 잘게:

단계	스텝	예 ($K_d=0.3$)
방향 잡기	$\times 1.5 / \times 0.67 (\pm 50\%)$	$0.3 \rightarrow 0.45$ 또는 0.2
좁혀가기	$\times 1.2 / \times 0.83 (\pm 20\%)$	$0.45 \rightarrow 0.54$ 또는 0.375
미세 마무리	$\times 1.1 / \times 0.9 (\pm 10\%)$	$0.5 \rightarrow 0.55$ 또는 0.45

규칙: 한 번에 하나만, 같은 코스를 반복 주행하며 비교. 둘 다 동시에 바꾸면 뭐가 효과인지 모릅니다.

증상별 손잡이

증상	조정
라인 위 갈지자/진동	$K_d \uparrow$ (먼저), 그래도면 $K_p \downarrow$
곡선에서 라인 못 따라가고 느림(lag)	$K_p \uparrow$
K_d 를 너무 올려 부르르 떨림/소음	$K_d \downarrow$ (노이즈 증폭 — 미분의 부작용)
곡선/급이탈에서 바깥으로 흘러나감	$PID_MAX_CORRECTION \uparrow$
직선에서 좌우 지그재그	$PID_MAX_CORRECTION \downarrow$

$PID_MAX_CORRECTION$ (Settings_robot1.h:113)은 보정량 saturation. $leftPWM = base + correction$, $rightPWM = base - correction$ 이라 좌우 PWM 차이는 최대 $2 \times$ 이 값. 주 역할은 D항 순간 스파이크 억제.

한 줄 요약

- P = 지금 위치, D = 변하는 속도(브레이크), I = 누적(이 로봇엔 불필요).
- $K_d \gg K_p$ 인 건 $dError$ 가 $error$ 보다 작아서(스케일 보정) + 진동을 세게 눌러야 해서(댐핑).
- 튜닝은 곱셈 스텝으로, 큰 폭 $\rightarrow$ 작은 폭, 한 번에 하나만.